

Staged source edition | 11 September 2026 | Evidence and authority remain distinct

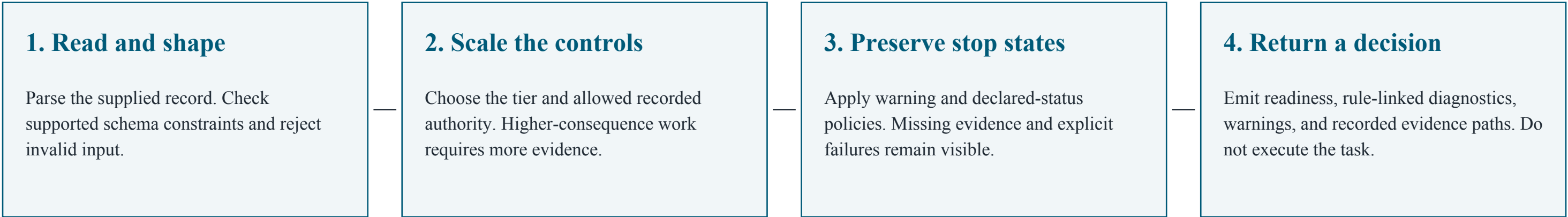
Staged source edition | 11 September 2026 | Evidence and authority remain distinct

# DEGS | what the core does

A deterministic evaluator for engineering task records. Current corrected source: 1,213 lines.



**Figure 1**  
*The evaluator's main path*



## Why the boundaries matter

A source path is not proof that its contents are correct. A recorded approval is not an approval granted by the program. The code checks the structured claim against its rules; people retain responsibility for consequential evidence and action.

The staged gate includes a bounded JSON equality correction; policy rules and canonical fixture outcomes are preserved. Local activation state, private audit history, and the Workbench adapter are outside this draft. The Sashiko-derived concern collections are a separate component, not part of this gate path.

# DEGS | function map

The complete source plate preserves line coverage. This map identifies each top-level unit and its role.



|         |                              |                                                                                                 |
|---------|------------------------------|-------------------------------------------------------------------------------------------------|
| 92-97   | nonempty                     | Rejects null, false, and empty strings or containers; other values count as present.            |
| 100-101 | nonblank_string              | Requires a string containing at least one non-whitespace character.                             |
| 104-109 | nonblank_string_list         | Requires at least one string and rejects blank or non-string entries.                           |
| 112-116 | status_is                    | Compares an object's status, converted to uppercase, against allowed labels.                    |
| 119-124 | status_with_evidence_path_is | Requires an exact allowed status and a nonblank evidence-path string.                           |
| 127-139 | test_category_passes         | Requires every recorded test in a named category to claim PASS and provide an evidence path.    |
| 142-170 | deterministic_controls_valid | Checks recorded external boundaries, enforcement reasons, denial tests, and follow-up evidence. |
| 173-197 | artifact_identity_valid      | Checks matching recorded artifact identifiers and a declared current evidence basis.            |
| 200-213 | risk_analysis_valid          | Requires a passing recorded analysis covering every defined risk factor.                        |
| 216-234 | change_traceability_valid    | Checks recorded requirement-to-artifact-to-test links and a declared zero orphan count.         |
| 237-243 | explicit_non_applicability   | Accepts a recorded non-applicable status only with false applicability and a reason.            |
| 246-280 | data_governance_valid        | Checks recorded data purpose, access, retention, authority, and any planned repurposing.        |
| 283-294 | threat_model_valid           | Checks a declared current threat model or a reasoned non-applicability record.                  |
| 297-311 | monitoring_valid             | Checks recorded ownership, thresholds, reassessment, and degradation or non-applicability.      |
| 314-328 | measurable_ai_criteria_valid | Requires measures, thresholds, and evidence paths in all four AI criteria categories.           |
| 331-366 | ai_assurance_valid           | Checks recorded consequential-AI controls and tests or reasoned non-applicability.              |

Imports, constants, gaps between definitions, and the final entry-point guard remain visible in the complete source plate. Source labels are explanatory; the repository source is authoritative.

# DEGS | function map

The complete source plate preserves line coverage. This map identifies each top-level unit and its role.



|         |                                         |                                                                                                     |
|---------|-----------------------------------------|-----------------------------------------------------------------------------------------------------|
| 369-409 | <code>hazard_analysis_valid</code>      | Checks recorded hazard controls, tests, and any human acceptance of residual risk.                  |
| 412-425 | <code>independent_review_valid</code>   | Requires a named reviewer distinct from the author, issue disposition, and passing retest evidence. |
| 428-429 | <code>add_unmet</code>                  | Appends one unmet requirement with its field, rule identifier, and explanation.                     |
| 432-443 | <code>collect_evidence_paths</code>     | Recursively gathers unique nonempty evidence-path strings without opening their files.              |
| 446-456 | <code>contains_secret_like_field</code> | Recursively flags prohibited field names in dictionaries and lists.                                 |
| 459-468 | <code>result</code>                     | Builds a decision record with unmet requirements, warnings, rule identifiers, and evidence paths.   |
| 471-485 | <code>invalid_result</code>             | Builds an input-failure record and returns the input-error exit code 4.                             |
| 488-500 | <code>load_task</code>                  | Reads a task object and checks its version plus prohibited telemetry or state field names.          |
| 503-525 | <code>json_values_equal</code>          | Compares nested JSON with a worklist, preserving numeric equality and separating booleans.          |
| 528-535 | <code>json_type_matches</code>          | Recognizes object, array, string, boolean, and null; other type labels do not match.                |
| 538-539 | <code>child_schema_path</code>          | Builds a dotted field path for a nested validation finding.                                         |
| 542-551 | <code>resolve_local_schema_ref</code>   | Resolves a local JSON pointer and rejects external or missing schema references.                    |
| 554-675 | <code>json_schema_errors</code>         | Evaluates selected schema keywords recursively and returns field-specific errors.                   |
| 678-680 | <code>schema_rule_for_path</code>       | Maps a field's top-level name to its DEGS rule identifier.                                          |
| 683-710 | <code>validate_structure</code>         | Loads the fixed task schema and turns distinct structural errors into a PASS or FAIL record.        |
| 713-750 | <code>evaluate_tier0</code>             | Checks recorded read-only scope, objective, secret handling, and Command Center lane.               |

Imports, constants, gaps between definitions, and the final entry-point guard remain visible in the complete source plate. Source labels are explanatory; the repository source is authoritative.

# DEGS | function map

The complete source plate preserves line coverage. This map identifies each top-level unit and its role.



|           |                              |                                                                                                 |
|-----------|------------------------------|-------------------------------------------------------------------------------------------------|
| 753-806   | tier1_controls               | Collects missing scope, requirements, baseline, tests, rollback, handoff, and lane evidence.    |
| 809-811   | evaluate_tier1               | Returns BLOCKED when Tier 1 has unmet controls; otherwise returns PASS.                         |
| 814-943   | tier2_controls               | Extends Tier 1 with recorded recovery, risk, identity, review, and operational evidence checks. |
| 946-948   | evaluate_tier2               | Returns BLOCKED when Tier 2 has unmet controls; otherwise returns PASS.                         |
| 951-963   | unresolved_critical_warning  | Finds unresolved critical entries among warnings and unresolved items.                          |
| 966-984   | unresolved_warning_summaries | Summarizes unresolved warning severities without copying their supplied messages.               |
| 987-1004  | apply_warning_policy         | Changes a passing result to BLOCKED for critical warnings or PASS_WITH_WARNINGS otherwise.      |
| 1007-1023 | apply_declared_status_policy | Stops an otherwise passing task when its declared status is FAILED or BLOCKED.                  |
| 1026-1114 | evaluate_tier3               | Adds recorded human approval, audit, hazards, recovery, target, and dry-run checks to Tier 2.   |
| 1117-1139 | evaluate                     | Selects the tier checks, fails unsupported tiers, and applies the warning policy.               |
| 1142-1158 | render_human                 | Formats the decision, unmet requirements, warnings, rules, and evidence paths for reading.      |
| 1161-1169 | parse_args                   | Parses validate, evaluate, or explain with a task path and optional JSON output.                |
| 1172-1209 | main                         | Loads and checks a task, prevents schema-invalid passes, then emits the result and exit code.   |

Imports, constants, gaps between definitions, and the final entry-point guard remain visible in the complete source plate. Source labels are explanatory; the repository source is authoritative.

# DEGS | the defect the public examples exposed

Fourteen manifestations. One comparator defect. A tested correction with its history preserved.

---



## The distinction must survive nesting

JavaScript Object Notation (JSON) distinguishes booleans from numbers. The corrected helper preserves that distinction through arrays and objects. A const schema requiring [false] now rejects [0], while 1 and 1.0 remain equal. The first implementation delegated container equality to Python; all 14 original failures came from that defect.

## What the independent review established

All 14 invalid labels and false acceptances were reproduced from the pinned public corpus: six enum cases and eight const cases. The current canonical schema has 53 const nodes and 16 enum nodes, with no container-valued comparison operands. Four scalar boolean/number substitutions failed structure checks. No current canonical task bypass was demonstrated.

## What the correction preserves

An explicit worklist compares nested arrays and objects without consuming a Python call frame per level. Booleans remain distinct from numbers; 1 and 1.0 remain equal. Array order and length matter; object member order does not. Independent review caught a depth regression in the intermediate correction. New 600-level array/object tests now pass (Wright et al., 2022).

Benchmark: 241/241 matched in the unchanged supported subset. This is helper interoperability evidence, not full JSON Schema conformance or whole-system accuracy. The original 227/241 observation remains intact. Harness corrections preserve timeout diagnostics, require stale freshness, and record actual invocation arguments.

# Source license

The DEGS source listing retains the repository's original license notice.

---

MIT License

Copyright (c) 2026 tmccoy678

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

# References

---

- Hirshorn, S. R., Voss, L. D., & Bromley, L. K. (2017). *NASA systems engineering handbook* (NASA/SP-2016-6105 Rev. 2). National Aeronautics and Space Administration. <https://ntrs.nasa.gov/citations/20170001761>
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST Special Publication 800-218). <https://doi.org/10.6028/NIST.SP.800-218>
- Wright, A., Andrews, H., Hutton, B., & Dennis, G. (2022, June 16). *JSON Schema: A media type for describing JSON documents* (Internet-Draft draft-bhutton-json-schema-01, Section 4.2.2; work in progress). <https://json-schema.org/draft/2020-12/json-schema-core#section-4.2.2>